Overview of Monte Carlo tree search

Subject: Monte Carlo tree search

1.

Selection: Start from root R and select successive child nodes until a leaf node L is reached. The root is the current game state and a leaf is any node that has a potential child from which no simulation (playout) has yet been initiated. The section below says more about a way of biasing choice of child nodes that lets the game tree expand towards the most promising moves, which is the essence of Monte Carlo tree search. Expansion: Unless L ends the game decisively (e.g. win/loss/draw) for either player, create one (or more) child nodes and choose node C from one of them. Child nodes are any valid moves from the game position defined by L. Simulation: Complete one random playout from node C. This step is sometimes also called playout or rollout. A playout may be as simple as choosing uniform random moves until the game is decided (for example in chess, the game is won, lost, or drawn). Backpropagation: Use the result of the playout to update information in the nodes on the path from C to R.

2.

Advantages and disadvantages Although it has been proven that the evaluation of moves in Monte Carlo tree search converges to minimax when using UCT, the basic version of Monte Carlo tree search converges only in so called "Monte Carlo Perfect" games. However, Monte Carlo tree search does offer significant advantages over alpha–beta pruning and similar algorithms that minimize the search space. In particular, pure Monte Carlo tree search does not need an explicit evaluation function. Simply implementing the game's mechanics is sufficient to explore the search space (i.e. the generating of allowed moves in a given position and the game-end conditions). As such, Monte Carlo tree search can be employed in games without a developed theory or in general game playing. The game tree in Monte Carlo tree search grows asymmetrically as the method concentrates on the more promising subtrees. Thus, it achieves better results than classical algorithms in games with a high branching factor. A disadvantage is that in certain positions, there may be moves that look superficially strong, but that actually lead to a loss via a subtle line of play. Such "trap states" require thorough analysis to be handled correctly, particularly when playing against an expert player; however, MCTS may not "see" such lines due to its policy of selective node expansion. It is believed that this may have been part of the reason for AlphaGo's loss in its fourth game against Lee Sedol. In essence, the search attempts to prune sequences which are less relevant. In some cases, a play can lead to a very specific line of play which is significant, but which is overlooked when the tree is pruned, and this outcome is therefore "off the search radar".

3.

Domain-specific knowledge may be employed when building the game tree to help the exploitation of some variants. One such method assigns nonzero priors to the number of won and played simulations when creating each child node, leading to artificially raised or lowered average win rates that cause the node to be chosen more or less frequently, respectively, in the selection step. A related method, called progressive bias, consists in adding to the UCB1 formula a b i n i {\displaystyle {\frac {b_{i}}{n_{i}}}} element, where bi is a heuristic score of the i-th move. The basic Monte Carlo tree

search collects enough information to find the most promising moves only after many rounds; until then its moves are essentially random. This exploratory phase may be reduced significantly in a certain class of games using RAVE (Rapid Action Value Estimation). In these games, permutations of a sequence of moves lead to the same position. Typically, they are board games in which a move involves placement of a piece or a stone on the board. In such games the value of each move is often only slightly influenced by other moves. In RAVE, for a given game tree node N, its child nodes Ci store not only the statistics of wins in playouts started in node N but also the statistics of wins in all playouts started in node N and below it, if they contain move i (also when the move was played in the tree, between node N and a playout). This way the contents of tree nodes are influenced not only by moves played immediately in a given position but also by the same moves played later.

4.

Pure Monte Carlo game search This basic procedure can be applied to any game whose positions necessarily have a finite number of moves and finite length. For each position, all feasible moves are determined: k random games are played out to the very end, and the scores are recorded. The move leading to the best score is chosen. Ties are broken by fair coin flips. Pure Monte Carlo Game Search results in strong play in several games with random elements, as in the game EinStein würfelt nicht!. It converges to optimal play (as k tends to infinity) in board filling games with random turn order, for instance in the game Hex with random turn order. DeepMind's AlphaZero replaces the simulation step with an evaluation based on a neural network.

5.

Leaf parallelization, i.e. parallel execution of many playouts from one leaf of the game tree. Root parallelization, i.e. building independent game trees in parallel and making the move basing on the root-level branches of all these trees. Tree parallelization, i.e. parallel building of the same game tree, protecting data from simultaneous writes either with one, global mutex, with more mutexes, or with non-blocking synchronization.

6.

Improvements Various modifications of the basic Monte Carlo tree search method have been proposed to shorten the search time. Some employ domain-specific expert knowledge, others do not. Monte Carlo tree search can use either light or heavy playouts. Light playouts consist of random moves while heavy playouts apply various heuristics to influence the choice of moves. These heuristics may employ the results of previous playouts (e.g. the Last Good Reply heuristic) or expert knowledge of a given game. For instance, in many Go-playing programs certain stone patterns in a portion of the board influence the probability of moving into that area. Paradoxically, playing suboptimally in simulations sometimes makes a Monte Carlo tree search program play stronger overall.